

Cloud Computing and Virtualization

Bachelors in Computer Engineering

**Prepared by Er. Ganga Gautam,
Assistant Professor, Pokhara University**

Chapter 2

Virtualization(7 hrs)

Outlines:

1. Introduction

- 1.1. Virtualization Technologies: Virtual machines, Hypervisors (Type 1 & Type 2)

- 1.2. overview of Containerization and Kubernetes

2. Virtual Machine (VM)

- 2.1. VM management and configuration

3. Virtualization Level

- 3.1. Hardware, Software, Operating System, Data and Networking

4. Virtual Private Cloud (VPC)

5. Virtual Private Network (VPN)

6. DNS service and Cloud Network

7. Multihoming

Virtualization

- The process of creating a virtual (rather than physical) version of computing resources — such as servers, storage, network, or operating systems.
- Physical resources is divided into logical (virtual) resources.
- Enable Multiple environments run on a single physical hardware.
- **Goal:**
 - Efficient resource utilization & isolation.
- **Example:**
 - Running Windows and Linux on the same machine via VMware or VirtualBox

Why Virtualization?

- **Before Virtualization:**

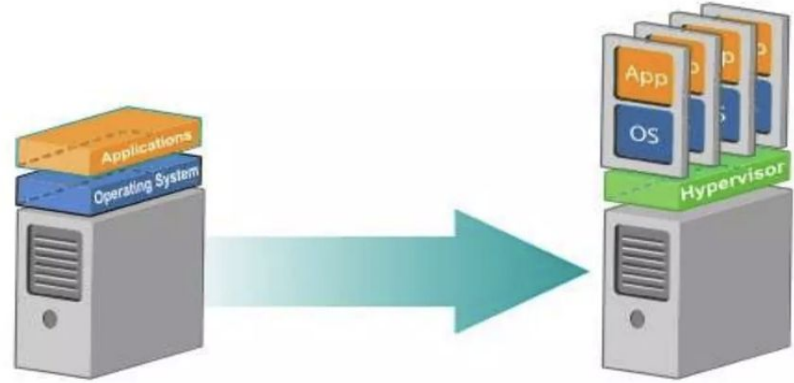
- One OS per machine
- Underutilized resources (CPU idle time)
- High hardware & maintenance cost

- **After Virtualization:**

- Multiple OSs on one machine
- Efficient utilization of resources
- Easier provisioning, isolation, and testing

- **Example:**

- A single server can host 10 VMs for different applications instead of 10 physical servers.



History of virtualization

1964 IBM designs the **CP-40 mainframe** in response to Project MAC's need of a new type of computer hardware capable of supporting more than one simultaneous user.



1994 Marc Ewing creates Red Hat Linux.



1998 VMware is founded.

2001 VMware releases **ESX Server 1.0**, an enterprise-class, Type 1 hypervisor.



2006 Intel VT-x and AMD-V introduce limited hardware support.
Amazon creates **AWS** and introduces Elastic Compute Cloud.

2008 Beta version of Hyper-V is shipped with certain x86-64 editions of Windows Server 2008.



2010 Microsoft releases Microsoft Azure.

2013 Docker Inc. debuts its Docker container software.
Google releases Google Compute Engine.

2018 Amazon announces Firecracker Open Source Technology for micro VMs.
AWS announces AWS Outposts, which brings native AWS offerings, infrastructure and operating models to data centers.

2019 VMware acquires Bitfusion, which provides high-performance computing. AI and machine learning.

1967 The first hypervisors providing full virtualization, **SIMMON** and IBM's one-off research CP-40 system, begin production use.

1972 IBM adds virtual memory support for System/370.

1974 Gerald Popek and Robert Goldberg classify the hypervisor into two types: Type 1 and Type 2.



1999 VMware releases **Workstation**, a hosted hypervisor that runs on x64 versions of Windows and Linux OSes.

2003 VMware releases VMware VirtualCenter 1.0, which includes the first iteration of vMotion.
Xen, the first open source Type 1 hypervisor, is publicly released.

2004 VMware ESX becomes ESXi.

2007 Citrix acquires **XenSource** and its Xen Project.



KVM merges into the Linux kernel mainline in kernel version 2.6.20.
VMware introduces Storage vMotion.

2012 VMware releases vSphere 5.1, which includes vSphere Storage Appliance, vSphere Data Protection, vSphere Replication and vShield Endpoint.

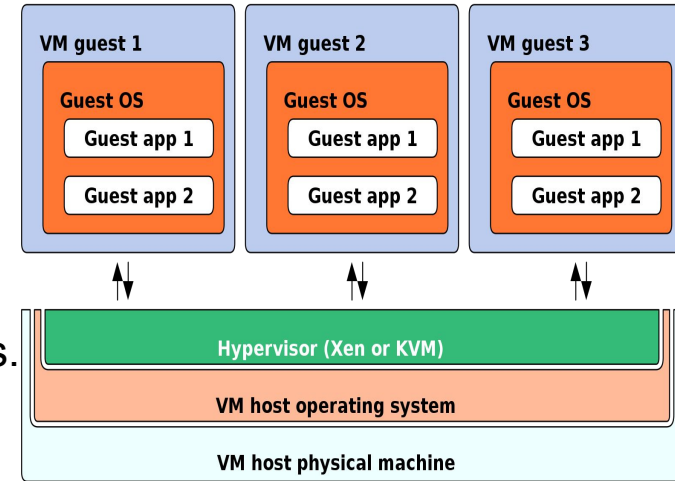
2017 VMware and AWS launch VMware Cloud on AWS, a SaaS delivering a vSphere compatible cloud in an AWS data center.

2019 VMware announces **Project Pacific**, which re-architects vSphere to deeply integrate and embed Kubernetes, and Tanzu, which provisions new clusters and attaches existing clusters running in multiple environments, such as vSphere, VMC and public clouds, to centralize management and operations.
IBM acquires Red Hat for \$34 billion.

Project Pacific

Basic Components of Virtualization

1. **Host Machine:**
 - Physical computer running virtualization software.
2. **Hypervisor:**
 - Software or firmware that creates and manages VMs.
3. **Guest Machine:**
 - Virtualized system with its own OS and applications.
4. **Virtual Resources:**
 - Virtual CPU, Memory, Disk, Network Interface.



Types of Virtualization

Server Virtualization

- Divides a physical server into multiple virtual servers, each with its own operating system and applications.
- Maximizes the use of server hardware, reduces costs, and improves efficiency.

Desktop Virtualization

- Separates a user's desktop environment from their physical device.
- Allows users to access their desktops remotely from various devices, which can be managed through methods like Virtual Desktop Infrastructure (VDI).

Storage Virtualization

- Pools physical storage from multiple devices into a single, centralized storage system.
- Simplifies management, increases storage capacity utilization, and makes disaster recovery easier.

Types of Virtualization (Contd...)

Network Virtualization

- Combines network resources into a single, software-based administrative entity.
- Creates virtual networks that operate independently of the physical hardware, which can improve performance, security, and organization.

Application Virtualization

- Separates applications from the underlying operating system so they can run in isolation.
- Applications can be delivered to end-user devices without needing to be installed locally.

Virtual Machines (VMs)

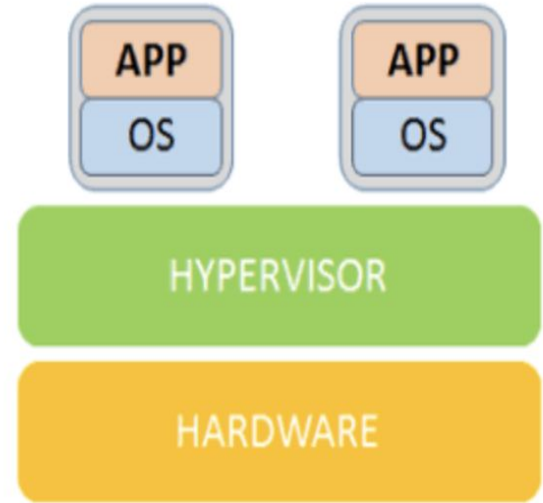
- Software-based emulation of a physical computer.
- Each VM includes:
 - Virtual hardware
 - Guest operating system
 - Applications
- VM Files:
 - .vmdk (disk), .vmx (config) in VMware
 - .vdi in VirtualBox
- **Advantages:**
 - Isolation,
 - security,
 - easy migration,
 - snapshot support.

Hypervisors

- A hypervisor is a software layer that enables virtualization by allowing multiple OSs to share hardware.
- Key Functions:
 - **Resource Scheduling:**
 - Allocates CPU, memory, and I/O resources among VMs.
 - **VM Lifecycle Management:**
 - Creates, runs, pauses, and deletes virtual machines.
 - **Isolation:**
 - Ensures each VM operates independently and securely.
 - **Performance Monitoring:**
 - Tracks resource usage and system health.
 - **Migration & Scheduling:**
 - Supports live migration and workload balancing.

Type 1 Hypervisor (Bare Metal)

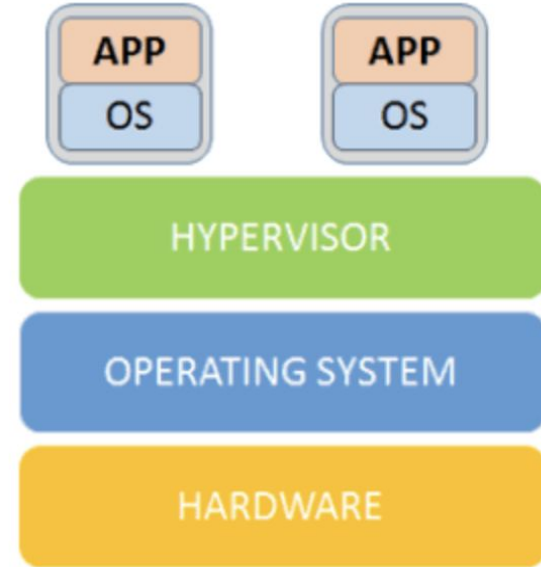
- Installed directly on hardware (no host OS).
- High performance, secure, used in data centers.
- Used in enterprise servers
- **Examples:**
 - VMware ESXi
 - Microsoft Hyper-V (Core)
 - XenServer
 - KVM
- **Advantages:**
 - High performance: direct access to the hardware
 - Secure: minimal surface attack
 - Stable and reliable for production environment.



Type 1 Hypervisor

Type 2 Hypervisor (Hosted)

- Runs on top of a host OS.
- Easier to install and manage, used for personal or test environments.
- **Examples:**
 - Oracle VirtualBox
 - VMware Workstation/Fusion
 - Parallels Desktop (macOS)
- **Advantages:**
 - User-friendly,
 - ideal for testing/lab use



Type 2 Hypervisor

Comparison: Type 1 vs Type 2

Feature	Type 1	Type 2
Installation	Direct on hardware	On top of host OS
Performance	High	Moderate
Security	High (isolated)	Lower (depends on host OS)
Use Case	Data centers, production	Development, testing
Examples	ESXi, KVM	VirtualBox, VMware Workstation

Problems Before Containerization

- **Environment Conflicts:**
 - “Works on my machine” — apps failed on other systems due to OS or library differences.
- **Slow Deployment:**
 - Setting up full environments or virtual machines took time and effort.
- **Resource Waste:**
 - Each VM needed its own OS → large, slow, and inefficient use of hardware.
- **Inconsistent Environments:**
 - Development, testing, and production setups often behaved differently.
- **Tight Coupling:**
 - Applications were monolithic — hard to update, scale, or move.

Containerization

- A packaging software code along with its required OS libraries and dependencies into a lightweight executable known as a *container*.
- Containers enable “write once, run anywhere” across on-premises, hybrid cloud and multi cloud environments.
- Rather than tying the application to a specific OS version or environment, you create a “container” that can run consistently on different machines and platforms.

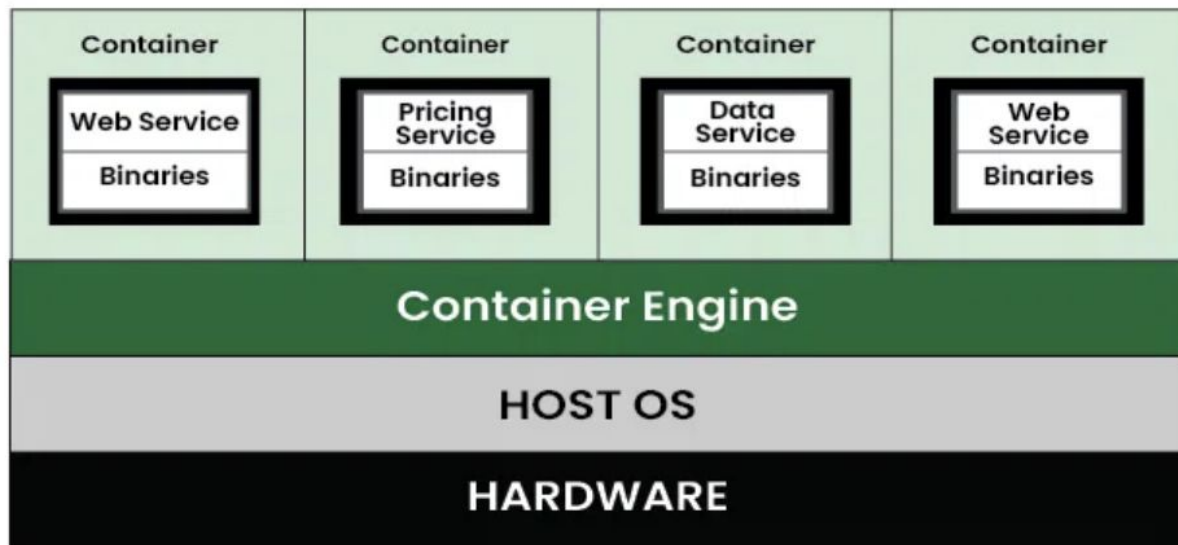
- Difference from VMs:
 - No separate OS per container
 - Lightweight and faster
 - Share same kernel
 - VM virtualizes hardware, container virtualizes OS.
- **Tools:**
 - Docker,
 - Podman,
 - LXC
- **Key Features:**
 - Fast startup time
 - Lower resource usage
 - Portable across environments

Containers vs Virtual Machines

Criteria	Containers	Virtual Machines
Abstraction	OS-level	Hardware-level
Boot Time	Seconds	Minutes
Size	MBs	GBs
Isolation	Process-level	Full OS-level
Management	Docker, Kubernetes	Hypervisors
Use Case	Microservices, CI/CD	Full OS virtualization

Use Cases:

- **Cloud migration (lift-and-shift):**
 - Encapsulating legacy applications into containers and moving them to the cloud without rewriting everything.
- **Microservice architectures:**
 - When you break an application into many small services, each can be containerized and managed independently.
- **IoT devices:**
 - Lightweight containers make deployment across devices with limited resources easier.



Container Orchestration

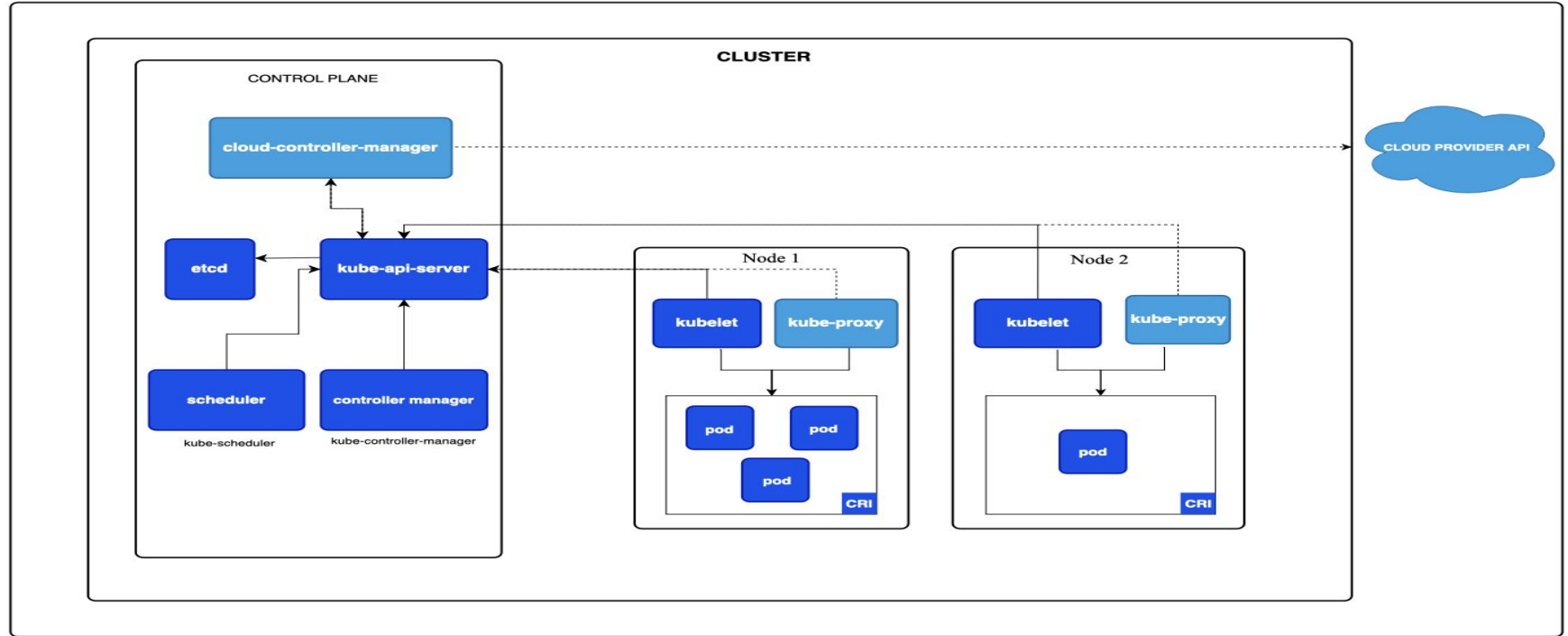
- Software technology for automated management of containers
- Handles deployment, scaling, networking, and monitoring of containers
- Essential for modern cloud-native apps with thousands of microservices
- Eliminates the need for manual container management
- **Why container orchestration???**
 - Applications today are made of many containerized microservices
 - Manual management is complex and error-prone
 - Orchestration ensures consistency, automation, and scalability across environments

Kubernetes

- Open-source system for container orchestration — automates deployment, scaling, and management of containerized applications.
- It supports running containers across multiple hosts/clusters, managing them as a unified system.
- Initially developed by Google (based on their “Borg” cluster manager experience) and now maintained by the Cloud Native Computing Foundation (CNCF).

- **Core Components:**
 - **Clusters:** Groups of machines (nodes) that Kubernetes manages.
 - **Control plane:** The “brain” of Kubernetes that makes scheduling decisions, monitors state.
 - **Nodes:** Worker machines where containers (Pods) run.
 - **Pods:** The smallest deployable units in Kubernetes (one or more containers).
 - **Deployments / ReplicaSets:** Abstractions that manage how many pods should run, and ensure they are maintained.
- **Benefits:**
 - Scalability,
 - fault tolerance,
 - self-healing,
 - automation

Architecture of Kubernetes: cluster architecture



Two-main zones: Control Plane & Worker Nodes

- A cluster is made up of a **control plane** (brains) + one or more **worker nodes** (machines running workloads).
- The control plane manages the state of the cluster, scheduling, coordination etc.
- Worker nodes run the actual containerized applications (via Pods).

Control Plane Components

Some of the key pieces:

- **kube-apiserver**: The front end of the control plane; exposes the Kubernetes API.
- **etcd**: Distributed key-value store that holds the cluster state.
- **kube-scheduler**: Decides which node should run a new Pod.
- **kube-controller-manager**: Runs controller processes (node controller, job controller, etc).
- **cloud-controller-manager**: Manages cloud-provider specific logic (if cluster is on a cloud).

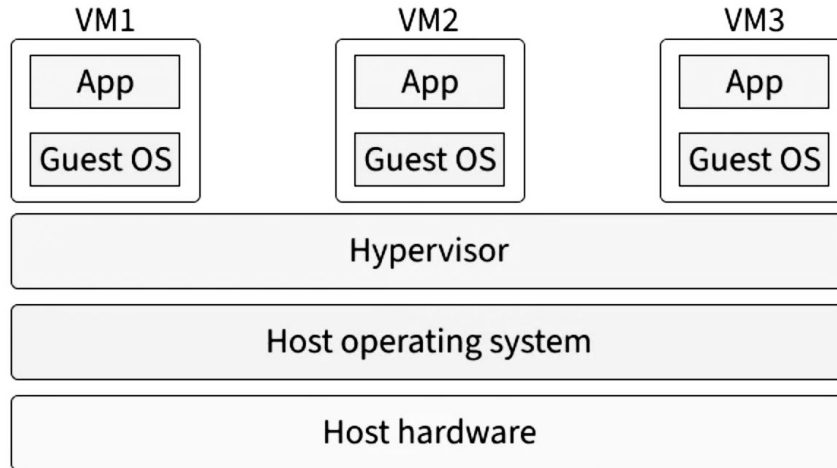
Worker Node Components

Each node has key functions:

- **kubelet**: Agent on each node that makes sure containers (Pods) are running as expected.
- **Container Runtime**: Software that actually runs containers (e.g., containerd, Docker) on the node.
- **kube-proxy**: Handles network rules on the node so that Services and Pods can communicate.

Virtual Machine (VM)

- A Virtual Machine (VM) is a software emulation of a physical computer.
- Runs its own OS and applications — independent of host.
- Contains virtual hardware components:
 - vCPU, vMemory, vDisk, vNIC (Network Interface Card).



VM Architecture

1. **Physical Layer:**
 - Hardware — CPU, RAM, Storage, NIC.
2. **Hypervisor Layer:**
 - Allocates and schedules resources.
3. **Virtual Hardware Layer:**
 - Virtual CPU, RAM, disk, NIC exposed to Guest OS.
4. **Guest OS:**
 - Installed inside the VM.
 - runs user applications
5. **Applications:**
 - Run on top of Guest OS.

Types of Virtual Machines

1. System Virtual Machines

- These are the most common type — what you get when you create a VM in VMware, VirtualBox, or AWS EC2.
- To virtualize an entire operating system (guest OS) on top of a host system.
- **Examples:**
 - Windows or Linux VM running inside VirtualBox
 - EC2 instance on AWS
 - VMware Workstation / Hyper-V VMs
- **Features:**
 - Each VM behaves like a full physical computer
 - Has its own CPU, memory, disk, and network interface
 - Managed by a hypervisor (e.g., KVM, VMware ESXi, Hyper-V)

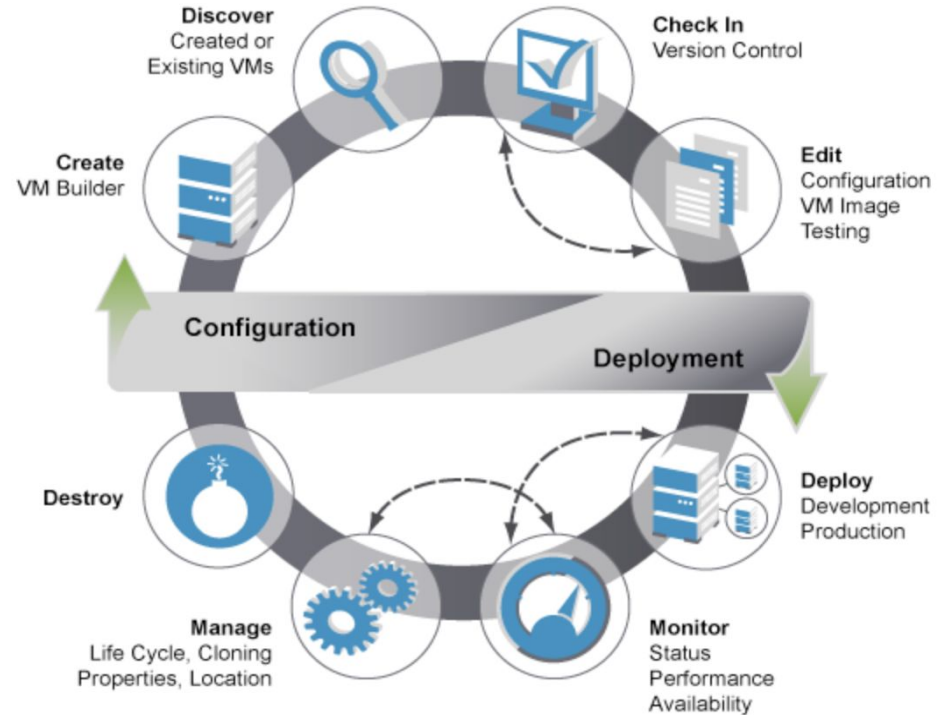
2. Process Virtual Machines

- These run a single program or process — not an entire OS.
- To provide a platform-independent runtime for applications.
- **Examples:**
 1. Java Virtual Machine (JVM) → runs Java bytecode on any OS
 2. .NET CLR (Common Language Runtime) → runs .NET programs
 3. Python Virtual Machine (interpreter)
- **Features:**
 - Designed for portability, not full system isolation
 - Ends when the application stops
 - Doesn't virtualize hardware — only the runtime environment

VM Lifecycle Management

Two Main Phases

- **Configuration Phase**
 - Build, version, and prepare VM images in a non-production environment.
- **Deployment Phase**
 - Run, monitor, manage, and retire VMs in production.



Configuration Phase Steps

- **Create**
 - Build VM using VM Builder (define CPU, RAM, disk, OS).
- **Discover**
 - Orchestrator Server auto-detects VMs with the Orchestrator Agent installed.
- **Check-In**
 - Store VM images into the VM Warehouse for tracking & version control.
- **Edit / Update**
 - Check-out → modify → check-in → new version created.
 - Mark a stable version as Gold Master for deployment.

Deployment Phase Steps

- **Deploy**
 - Convert VM template into a deployable VM instance; clone as needed.
- **Monitor**
 - Use Monitoring Service to track VM health, performance, availability.
- **Manage**
 - Runtime actions: migrate VMs, patch, update properties, balance workloads.
 - Automatic migration if host degrades or fails.
- **Destroy**
 - Retire VMs; remove from listing and delete image from Warehouse.

VM Creation Process

Steps to Create a VM:

Steps generally include:

1. Select compute host or cluster
2. Choose guest OS
3. Assign CPU & memory
4. Allocate storage
5. Configure network
6. Finalize and power on VM

VM Templates

- A pre-configured master image of a virtual machine used to create multiple identical VMs.
 - Contains OS installation, patches, and sometimes pre-installed applications.
 - Stored in a read-only state to prevent accidental changes.
 - Ensures standardization across all deployed virtual machines.
- **Types of Templates**
 - **Full VM Template:**
 - Contains complete OS + app stack
 - **Thin Template:**
 - Minimal OS and basic configuration
 - **Golden Image:**
 - Highly standardized, approved configuration used across organization

VM Snapshots

- A snapshot is a point-in-time copy of a virtual machine's disk, memory, and system state.
 - Disk State: Saves the exact contents of the virtual disk at the moment of snapshot.
 - Memory State (Optional): Captures RAM content → restores machine to running state instantly.
 - System State: Includes VM configuration, power state, and device state.
- It allows you to revert the VM back to an earlier condition if something goes wrong.
 - After a snapshot, VM writes changes to a delta (child) disk, preserving the original disk.
 - Multiple snapshots form a snapshot tree.
 - Restoring discards current changes and reverts to saved state.

VM Cloning

- The process of creating an exact copy of an existing virtual machine.
- Used for rapid deployment, testing, training labs, and scaling environments.
- **Types:**
 - **Full Clone:**
 - Independent copy of VM
 - **Linked Clone:**
 - Dependent copy sharing the base disk
 - **Instant Clone (vSphere 7.0+):**
 - Clone running VM in seconds

VM Migration

- The process of moving a virtual machine from one physical host or datastore to another without disrupting its operation.
- Ensures performance, availability, and flexibility in cloud environments.
- **Types:**
 - Cold Migration:
 - Move VM when it's powered off.
 - Live Migration:
 - Move running VM without downtime.
 - Storage Migration:
 - Relocate VM disk files to new datastore.

- **Benefits:**
 - Load Balancing
 - Hardware Maintenance
 - Performance Optimization
 - Error Avoidance
 - Resource Reallocation
- **Example:** migrating workloads between physical hosts in a VMware cluster during maintenance windows.

VM Monitoring

- The process of continuously observing and analyzing the performance, health, and resource usage of virtual machines (VMs)
- Ensures VM run efficiently, securely, and without interruption.

1. CPU Monitoring

- Tracks vCPU usage, ready time, and CPU contention
- Identifies overloaded hosts or misconfigured vCPU assignments
- Helps detect CPU bottlenecks due to resource overcommitment

2. Memory Monitoring

- Monitors vRAM usage, ballooning, swapping, and memory pressure
- Detects memory starvation, leaks, and excessive swapping
- Ensures proper allocation of memory shares and reservations

3. Storage (Disk I/O) Monitoring

- Measures disk read/write throughput and latency
- Identifies slow storage performance, I/O bottlenecks, or datastore congestion
- Helps determine when to migrate VMs to faster storage tiers

4. Network Monitoring

- Analyzes incoming/outgoing traffic, packet drops, and bandwidth usage
- Detects network saturation, misconfigured NICs, or connectivity issues
- Ensures optimal VM performance in distributed environments

5. Log & Event Monitoring

- Tracks system logs, application logs, and VM event history
- Helps detect failures, crashes, configuration issues, or security threats
- Supports auditing, troubleshooting, and compliance

6. Health & Availability Monitoring

- Monitors VM uptime, resource health, and status alerts
- Automatically detects failures or unusual behavior
- Integrates with alerting systems for proactive management

Monitoring Tools

- vSphere Performance Charts
- VMware vRealize Operations (vROps)
- Hyper-V Performance Monitor
- Proxmox Metrics
- CloudWatch (AWS), Azure Monitor, Stackdriver (GCP)

VM Management

- VM Management involves all activities required to deploy, operate, maintain, secure, and optimize virtual machines throughout their lifecycle.
- Key components include:

1. VM Provisioning

- Creating new VMs from templates or manually
- Selecting OS, CPU, memory, storage, and network settings
- Automating deployment using scripts or APIs

2. VM Configuration

- Assigning and adjusting vCPU, vRAM, storage size, and network adapters
- Setting BIOS/UEFI parameters
- Installing guest tools (VMware Tools/Guest Additions)
- Applying resource limits, reservations, and shares

3. Updating & Patch Management

- Applying OS updates, kernel patches, application patches
- Automating patch cycles using tools (WSUS, Ansible, SCCM, vSphere Lifecycle Manager)
- Ensuring compliance with security baselines

4. Monitoring & Performance Management

- Tracking CPU, memory, disk I/O, and network usage
- Detecting performance issues (resource contention, bottlenecks)
- Using monitoring tools (vCenter, CloudWatch, Azure Monitor, Prometheus)
- Setting up alerts and thresholds

5. Security & Access Management

- Implementing RBAC (Role-Based Access Control)
- Configuring firewall rules and network segmentation
- Enabling secure boot and disk encryption
- Hardening guest OS and applying security baselines

6. VM Lifecycle Operations

- Power operations: start, stop, pause, suspend
- Managing snapshots (create, revert, delete)
- Cloning, instant cloning, and migrations (vMotion)
- Backups and disaster recovery

7. Resource Optimization

- Avoiding over/under-provisioning
- Reclaiming unused storage (thin provisioning)
- Consolidating idle VMs and cleaning VM sprawl
- Balancing workloads across hosts (DRS, autoscaling)

VM Configuration Parameters

Resource	Description	Typical Range
-----	-----	-----
vCPU	Number of virtual processors	1–8 cores
vRAM	Virtual memory allocation	2 GB – 16 GB
vDisk	Virtual storage file	20 GB – 500 GB
vNIC	Virtual network adapter	NAT / Bridged / Internal
BIOS/EFI	Boot mode configuration	Legacy / UEFI

Storage Configuration

- **Virtual Disks:**
 - VMDK/VDI/VHD files that act as OS disks.
- **Disk Provisioning Types:**
 - **Thin Provisioning:** Allocates space as data grows (saves storage).
 - **Thick Provisioning – Lazy Zeroed:** Reserves full disk space but zeroing happens on first write.
 - **Thick Provisioning – Eager Zeroed:** Fully reserved and pre-zeroed (best performance).
- **Disk Modes:**
 - **Independent–Persistent:** Changes saved permanently.
 - **Independent–Non-persistent:** Changes discarded on shutdown (useful for testing).
- **Adding Storage:**
 - Add new virtual disks or extend existing ones without downtime (hot-add supported on many hypervisors).
- **Storage Policies:**
 - Define performance, RAID level, encryption, replication, and availability requirements.
- **Controller Types:**
 - SCSI, SATA, NVMe controllers depending on performance needs.

Network Configuration

- **Virtual NIC (vNIC):**
 - Connects VM to virtual switches and physical networks.
- **Network Adapters:**
 - Types: E1000, VMXNET3, VirtIO (varies by hypervisor).
- **IP Configuration:**
 - Static IP or DHCP from network environment.
- **VLAN Assignment:**
 - Tagging VM traffic for segmentation and security.
- **Multiple NICs:**
 - Multi-homing for redundancy or separating traffic types (mgmt, app, DB).
- **MAC Address Options:**
 - Automatically assigned or manually reserved.
- **Network Policies:**
 - Security groups / firewall settings
 - Promiscuous mode, forged transmits, MAC changes
- **Virtual Switch Types:**
 - Standard vSwitch, Distributed vSwitch (for advanced networking).

Guest OS Configuration

OS Installation:

- Use ISO, template, or image to install the guest OS.

Hostname & Identity Settings:

- Configure hostname, domain membership, identity profiles.

Time Synchronization:

- Sync clock with host or NTP server to maintain consistency.

OS-Level Networking:

- Configure DNS, gateway, subnet mask, routes.

Firewall & Security:

- Enable OS firewall, configure ports, disable unnecessary services.

File System Setup:

- Create partitions, format disks, mount volumes.

User & Permission Management:

- Create admin/user accounts, configure sudoers or ACL rules.

Driver Tools Installation:

- Install VMware Tools / Hyper-V Integration Services / Guest Additions for improved performance and functionality.

vSphere VM Management (vSphere 6.5+)

- Create VMs
- Configure hardware
- Power operations
- Manage snapshots
- Basic cloning

vSphere 7.0 VM Automation Improvements

- Advanced cloning
- Instant clone
- Lifecycle automation
- Network and identity customization
- API-driven VM lifecycle management

VM Security Management

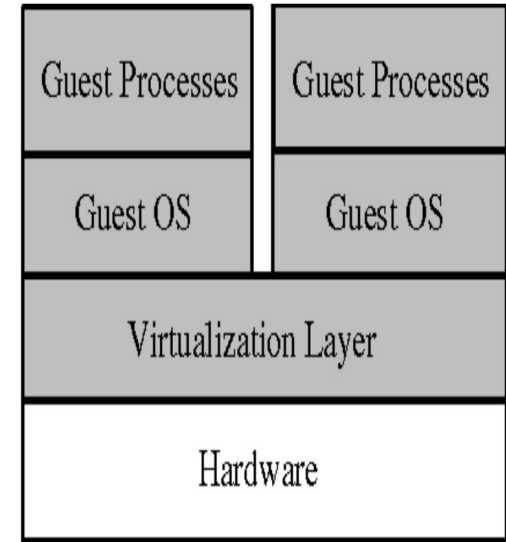
1. Role-Based Access Control (RBAC)
2. Hardening Templates
3. VM Disk Encryption
4. Secure Boot
5. Network Isolation
6. Firewall & Security Policies
7. Patch & Update Management
8. Logging & Monitoring
9. Antivirus / Endpoint Protection
10. Backup & Recovery Security

Virtualization Levels

- Virtualization can occur at multiple abstraction layers:
 - Hardware
 - Software
 - Operating System
 - Data
 - Networking
- Each level virtualizes different resources to enable flexibility, isolation, and scalability.
- Together, they form the foundation for modern cloud infrastructure.

Hardware-Level Virtualization (System Virtualization)

- Involves virtualizing the entire physical hardware of a system.
- Hypervisor (VMM) creates multiple Virtual Machines (VMs), each acting as an independent physical machine.
- VMM controls and monitors the processor, memory and other hardware resources.
- **Example:**
 - VMware ESXi, Microsoft Hyper-V, KVM.
- **Benefits:**
 - Hardware resource sharing
 - Fault isolation
 - Efficient hardware utilization

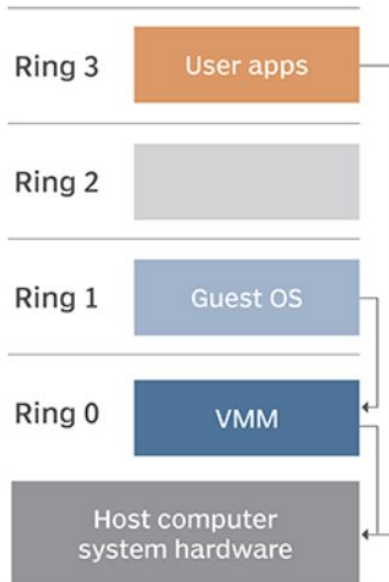


Hardware-level Virtualization

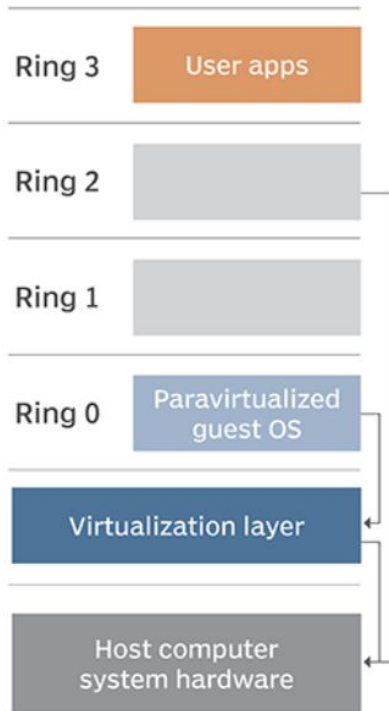
Hardware Virtualization Types

- **Full Virtualization:**
 - Guest OS runs unmodified.
 - Hypervisor emulates complete hardware.
 - Example: VMware Workstation.
- **Para-virtualization:**
 - Guest OS modified to communicate with hypervisor using special APIs.
 - Example: Xen, Oracle VM.
- **Hardware-assisted virtualization:**
 - Uses CPU extensions (Intel VT-x, AMD-V).
 - Improves performance and security.

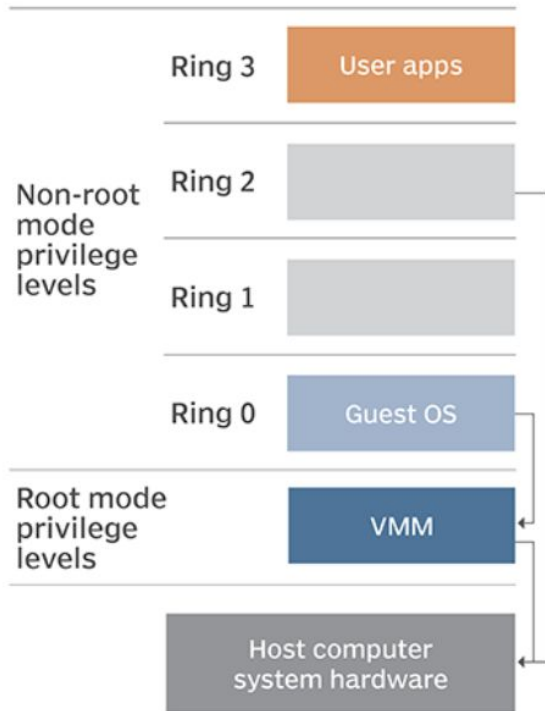
FULL VIRTUALIZATION



PARA VIRTUALIZATION



HARDWARE-ASSISTED VIRTUALIZATION

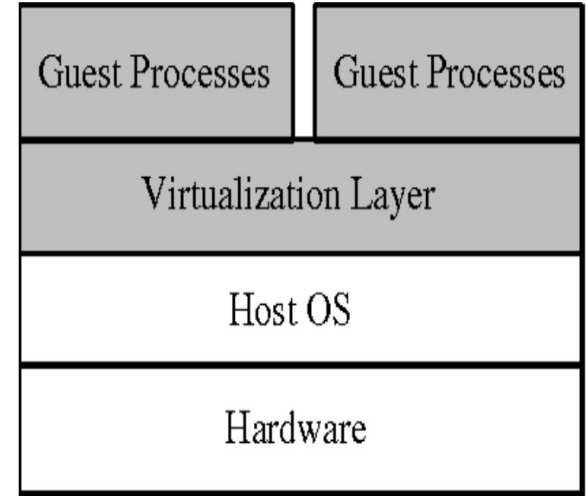


Software-Level Virtualization

- Virtualization happens at application or runtime environment level.
- Creates an isolated environment for applications without emulating entire hardware.
- Examples:
 - Java Virtual Machine (JVM)
 - .NET CLR (Common Language Runtime)
 - Wine (Windows emulator on Linux)
- Enables portability and sandboxing.

Operating System-Level Virtualization

- The OS kernel allows multiple isolated user-space instances.
- Each instances shares the same OS kernel but behaves independently.
- **Examples:**
 - Docker, LXC, Kubernetes Pods.
- **Advantages:**
 - Lightweight
 - Fast startup
 - Resource efficiency



OS-level Virtualization

Data Virtualization

- Abstracts data from its physical location and format to provide a unified, logical view.
- **Goal:**
 - Access and integrate data without moving or replicating it.
- **Key Components:**
 - Data abstraction layer
 - Data federation engine
 - Real-time integration
- **Benefits:**
 - Simplified access across heterogeneous sources
 - Reduced data duplication
 - Faster analytics

Networking Virtualization

- Abstracting physical network resources (switches, routers) into logical networks.
- Enables creation of virtual networks that can be managed like software.
- **Components:**
 - Virtual switches (vSwitch)
 - Virtual routers
 - Network overlays (VXLAN, GRE)
 - SDN (Software Defined Networking) controllers
- **Benefits:**
 - Dynamic network provisioning
 - Simplified management
 - Security and isolation

Integration of Virtualization Levels

- In real cloud environments, multiple levels coexist:
 - Hardware virtualization → virtual servers
 - OS virtualization → containers
 - Network virtualization → virtual networks
 - Data virtualization → unified data view
- Combined, they form Virtual Infrastructure (VI) used by IaaS, PaaS, SaaS layers.

Software Defined Networking (SDN)

Activity Suggestion:

Draw a multi-layer virtualization architecture diagram showing how containers, hypervisors, and SDN interact in a cloud setup.

Discuss: “Which virtualization level introduces the greatest security risk and why?”

Virtual Private Cloud

- A Virtual Private Cloud (VPC) is an isolated virtual network within a public cloud environment.
- Enables you to launch AWS resources in a virtual network that you define
- Gives control over virtual networking resources including:
 - Selection of IP address range
 - Creation of subnets
 - Configuration of route tables and network gateways
- Enables to customize the network configuration for your VPC
- Enables to use multiple layers of security
- **Examples:**
 - AWS VPC
 - Google Cloud VPC
 - Azure Virtual Network (VNet)

- A VPC is subdivided into subnets.
- A subnet belongs to one Availability Zone and requires a CIDR block.
- Route tables control traffic for a subnet.
- Route tables have a built in local route.
- You add additional routes to the table.
- The local route cannot be deleted.

Why VPC?

- **Need for VPC:**
 - Isolation and Security
 - Custom Network Configuration
 - Controlled Internet Access
 - Hybrid Connectivity (VPN, Direct Connect)
 - Regulatory Compliance

Traditional Network Vs VPC

Feature	VPC (Virtual Private Cloud)	On-Premise Networking
Infrastructure	Virtual, software-defined	Physical hardware
Scalability	Fast, on-demand	Slow, hardware-limited
Cost	Pay-as-you-go	High upfront cost
Management	Cloud provider handles most tasks	Requires in-house maintenance
Security/Isolation	Security Groups, NACLs	VLANs, physical firewalls
Provisioning	Minutes	Days/weeks
Flexibility	Easy to modify (IaC)	Rigid and manual

VPC architecture

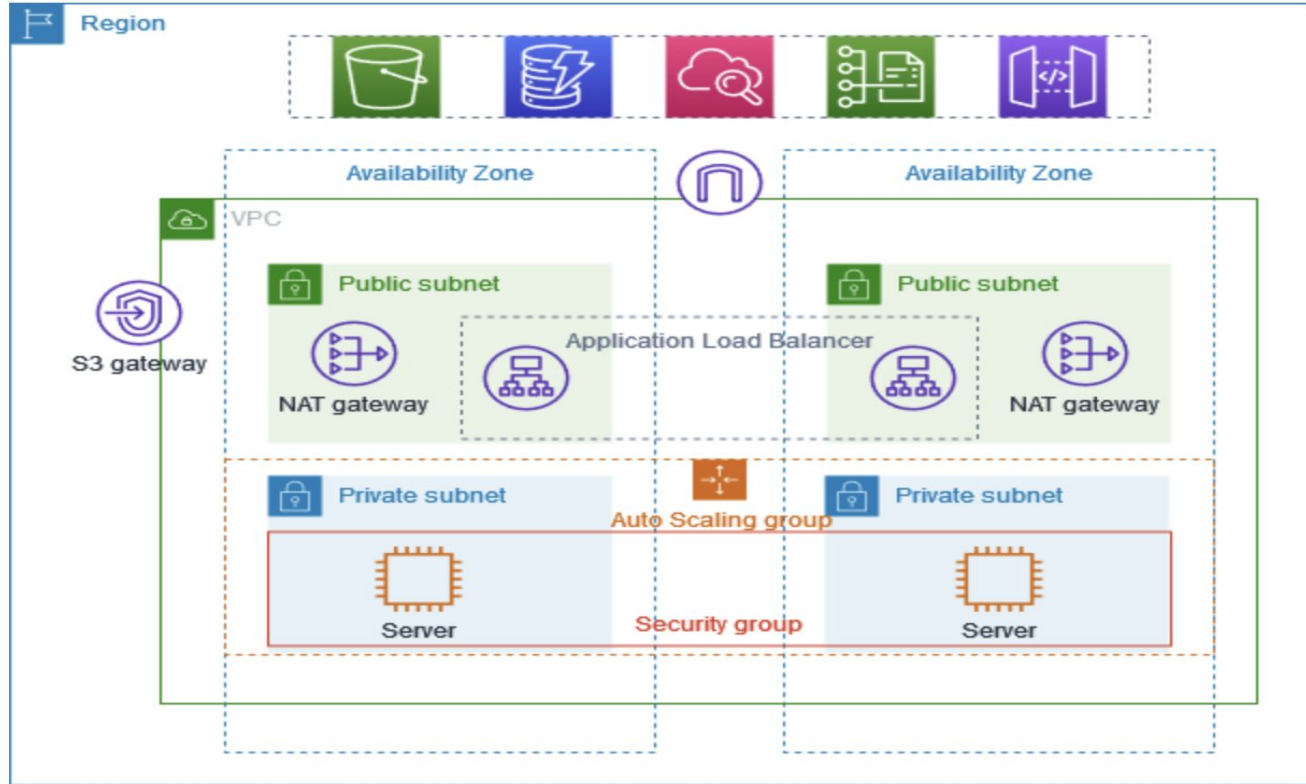


Figure: AWS VPC architecture diagram public private subnet

VPC Architecture

Key Components:

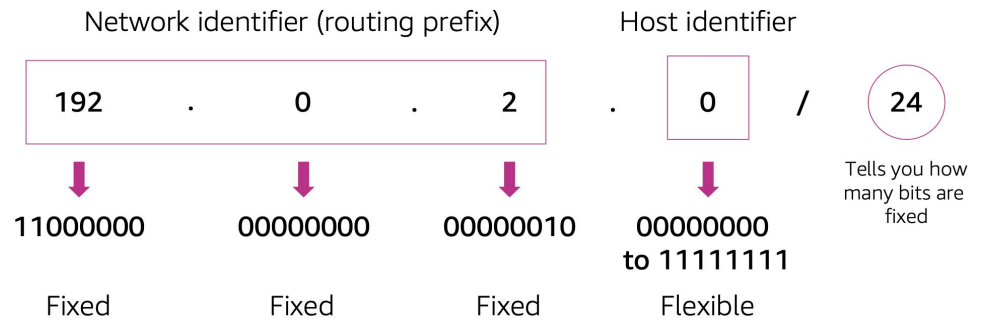
- Subnets
- Route Tables
- Internet Gateway (IGW)
- NAT Gateway
- Security Groups & NACLs
- DHCP Options Set:
 - Provides DNS and IP configuration

Customizing Network Configuration in a VPC

- **Subnet**

- Range of IP addresses that divide a VPC
- Belong to a single Availability Zone
- Classified as public or private
- **Public Subnet:**
 - Connected to Internet Gateway (IGW)
 - Create a Public Subnet
 1. Hosts web servers that need access to the public internet
 2. Enables inbound/outbound traffic for web applications
- **Private Subnet:**
 - No direct internet access, used for backend services
 - Create a Private Subnet
 1. Used for backend systems such as databases or application servers
 2. No direct internet access for enhanced security

VPC Subnets



- **CIDR Blocks:**

- Describes networks and IP address ranges efficiently,
Format: <IP Address>/<Prefix Length>
Example: 192.168.1.0/24
- The fixed bits define the network portion.
- The remaining bits can vary, representing hosts in that network.
- CIDR groups consecutive IP addresses into one block for efficient routing.

- **Subnet Masking Example:**

- VPC: 10.0.0.0/16
- Public subnet: 10.0.1.0/24
- Private subnet: 10.0.2.0/24

Routing in VPC

- **Route Tables:**

- Route table contains a set of rules (or routes) that you can configure to direct network traffic from your subnet.
- Each route specifies a destination and a target.
- By default, every route table contains a local route for communication within the VPC
- Each subnet must be associated with a route table (at most one)

Dynamic Routing:

Enabled through BGP in hybrid connections (VPN/Direct Connect)

Main (Default) Route Table

Destination	Target
10.0.0.0/16	local

VPC CIDR block

Internet Gateways

- A scalable, redundant, and highly available VPC component that enables communication between VPC instances and the internet.
- Highly available, horizontally scaled
- Main Purposes:
 - Provides a target in route tables for internet-routable traffic.
 - Performs Network Address Translation (NAT) for instances with public IPv4 addresses.
- Making a Subnet Public:
 - Attach an Internet Gateway to your VPC.
 - Add a route in the route table to direct non-local traffic (0.0.0.0/0) through the Internet Gateway.

NAT Gateways

- Enables instances in a private subnet to access the internet or AWS services, while blocking inbound connections from the internet.
- Configuration Steps:
 1. Create a NAT Gateway in a public subnet.
 2. Associate an Elastic IP address with the NAT Gateway.
 3. Update route tables for private subnets to send internet-bound traffic through the NAT Gateway.
- Key Benefits:
 - Managed service (no maintenance required)
 - High availability and scalability
 - Better performance than a NAT instance
- Use NAT Gateway instead of NAT Instance for most use cases.

Security in VPC

- Implement Multiple Layers of Security
 - Virtual firewalls controlling inbound/outbound traffic
 - **Security Layers:**
 - Security Groups
 1. control inbound and outbound traffic to EC2 instances
 2. Instance-level virtual firewall
 3. Stateful (allow return traffic automatically)
 4. Example: Allow port 22 (SSH), 80 (HTTP)
 - Network Access Control Lists (Network ACLs)
 1. provide an additional layer of subnet-level security
 2. Subnet-level control
 3. Stateless (explicit allow/deny both directions)
- **Best Practice:**
 - Use both SGs and NACLs for defense-in-depth.

Peering and Inter-VPC Connectivity

- A networking connection between two VPCs that enables private traffic routing between them.
 - Instances in either VPC communicate as if on the same network.
- Connection Options:
 - Between your own VPCs
 - With a VPC in another AWS account
 - Across different AWS Regions
- Restrictions:
 - Overlapping IP ranges not allowed
 - Transitive peering not supported
 - $(A \leftrightarrow B \text{ and } A \leftrightarrow C \text{ doesn't imply } B \leftrightarrow C)$
 - Only one peering connection allowed between the same two VPCs

Hybrid Cloud Connectivity

- **Site-to-Site VPN:**
 - IPsec-based encrypted tunnel between on-premise and VPC
 - By default, instances in a VPC cannot communicate with a remote (on-premises) network.
 - A Site-to-Site VPN connects your VPC to your corporate or remote network securely over the internet.
- **Direct Connect / ExpressRoute:**
 - Network performance can degrade when your data center is far from the AWS Region.
 - Solution – A dedicated, private network connection between your on-premises network and an Direct Connect location.
 - Lower latency, higher security

Virtual Private Network (VPN)

- A VPN creates a secure, encrypted tunnel between two or more endpoints over an untrusted network like the Internet.
- A VPN is a secure connection that links your on-premises network (e.g., office or data center) with your cloud environment (like AWS, Azure, GCP).
- It creates a private, encrypted tunnel over the internet so that cloud resources can be accessed safely.
- **Purpose:**
 - Protect data in transit
 - Ensure privacy and authentication
 - Connect remote users or sites securely
- Its “Private communication over public infrastructure”

Why VPN in Cloud Computing?

- Cloud workloads often span:
 - On-premises data centers
 - Cloud VPCs (AWS, Azure, GCP)
 - Remote developers or offices
- VPN enables:
 - Hybrid Cloud Integration
 - Secure Remote Access
 - Regulatory Compliance (e.g., GDPR, HIPAA)
- Example: Connecting enterprise LAN → AWS VPC via IPsec tunnel

Working of VPN

Core Mechanisms:

1. **Tunneling:**

Encapsulates packets within another packet to hide original data.
e.g., IP packet inside another IP packet.

2. **Encryption:**

Protects confidentiality and integrity (AES, DES).

3. **Authentication:**

Validates identity using certificates, PSKs, or credentials.

4. **Encapsulation Protocols:**

- PPTP, L2TP, IPsec, SSL/TLS, OpenVPN

How Cloud VPN Works

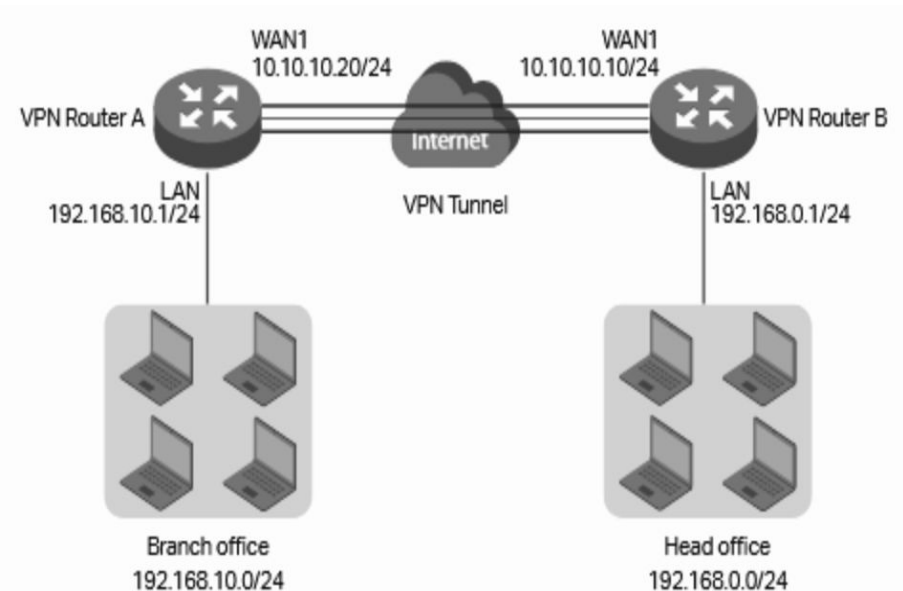
1. A VPN gateway is set up in the cloud (e.g., AWS VPN Gateway, Azure VPN Gateway).
2. A VPN device or firewall is configured in your on-premise network.
3. These two connect via:
 - IPsec VPN (most common)
 - SSL VPN

Now, A secure bridge between your data center and cloud.

VPN Architecture

Components:

- **VPN Client:**
 - Software on remote user/device
- **VPN Gateway:**
 - Hardware or virtual router handling encryption/decryption
- **VPN Tunnel:**
 - Encrypted path over public internet
- **Authentication Server:**
 - Verifies credentials or certificates



VPN Types

1. Remote Access VPN

- Lets a user connect to a private network over the Internet.
- Used by employees working from outside the office, or by home users.

2. Site-to-Site VPN

- Also called “Router-to-Router” VPN.
- Connects entire networks of different sites (like branch offices).
- Two subtypes:
 - Intranet-based VPN: between offices of the *same* company.
 - Extranet-based VPN: between different companies.

3. Cloud VPN

- Connects users (or on-premises networks) securely to cloud-based infrastructure.
- Uses the Internet as the transport medium.
- Usually uses standard encryption protocols like IPsec or SSL.
- Common for companies that use cloud services (e.g., AWS, Azure).

4. Mobile VPN

- Designed for mobile devices (phones, tablets).
- Works even when switching between different networks (Wi-Fi, cellular).
- Helps secure data when accessing private corporate resources.

VPN Protocols

Common VPN Protocols:

1. **PPTP (Point-to-Point Tunneling Protocol):**
 - Easy setup, weak encryption (obsolete)
2. **L2TP/IPsec (Layer 2 Tunneling Protocol):**
 - Encapsulation + encryption (IPsec)
3. **IPsec (Internet Protocol Security):**
 - Network-layer security, most used in enterprise and cloud VPNs
4. **SSL/TLS VPN:**
 - Operates at transport layer; supports clientless access
5. **OpenVPN:**
 - Open-source, flexible, uses SSL/TLS

VPN Deployment in Cloud

Cloud Provider Implementations:

- **AWS:** Site-to-Site VPN, Client VPN, Transit Gateway VPN
- **Azure:** Virtual Network Gateway
- **Google Cloud:** Cloud VPN (Classic & HA)

Common Setup:

1. Configure VPN Gateway on cloud side
2. Configure on-prem router/firewall (Cisco, FortiGate, etc.)
3. Exchange pre-shared keys / certificates
4. Verify tunnel via BGP or static routing

VPN Security Considerations

1. Encryption Strength

- Use AES-256 for data encryption
- Use SHA-2 for hashing and integrity

2. Key Management

- Rotate encryption keys regularly
- Avoid long-term reuse of the same keys

3. Split Tunneling

- Disable split tunneling unless absolutely required
- Reduces exposure by forcing all traffic through the tunnel

4. Access Control

- Integrate VPN authentication with IAM, RADIUS, or SSO
- Apply least-privilege access rules

5. Monitoring

- Use CloudWatch, Azure Monitor, or syslog to track:
 - Tunnel health
 - Connection attempts
 - Anomalous behavior

Limitations of VPN

1. Latency & Performance

- Encryption and decryption add processing overhead
- Can cause slower application performance

2. Scalability

- Difficult to manage many tunnels across locations
- More users = more load on VPN gateway

3. Single Point of Failure

- If the VPN gateway goes down, all connected users lose access
- Requires redundancy or failover setup

4. Complex Management

- Managing certificates, keys, routing rules, and device configurations increases operational complexity

Assignment

Configure a site-to-site VPN between an AWS VPC and an on-premises simulation using StrongSwan or Cisco Packet Tracer. Document steps, configurations, and encryption details.

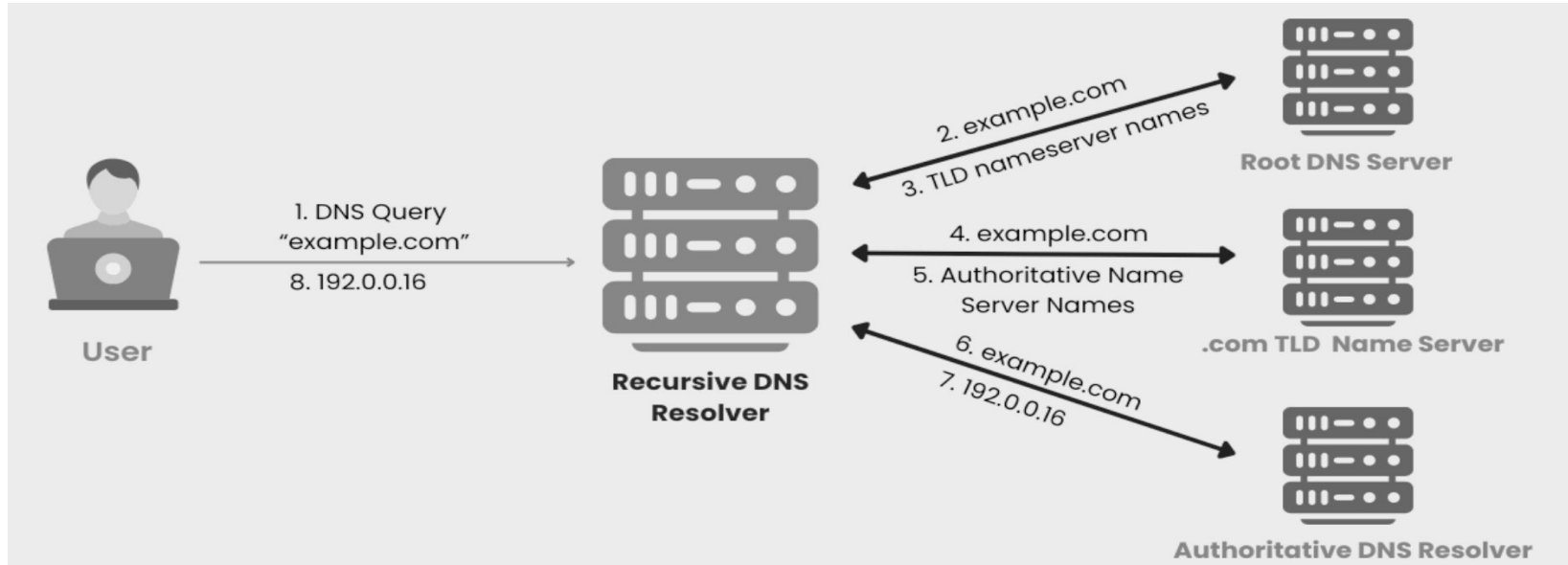
Domain Name System (DNS)

- Acts like the phonebook of the Internet
- Translates domain names (e.g., google.com) into IP addresses (e.g., 142.250.190.78)
- Without DNS, users would have to remember IP addresses
- Translates human-readable domain names into machine-readable IP addresses
- **Purpose:**
 - Simplifies user access
 - Enables scalability and redundancy
 - Foundation for distributed services
- Eg: Domain name: www.example.com
IP address 192.168.1.1

- Command to view IP address of Specific Domain

>nslookup

>Domain name



DNS Architecture:Types of DNS Servers

- **Recursive DNS Resolver:**
 - Receives queries from client machines and performs the necessary lookups to resolve a domain name into an IP address .
- **Root Name Server:**
 - The first step in translating human-readable domain names into IP addresses. It directs queries to the appropriate TLD server.
- **TLD Name Server:**
 - Responsible for storing the address information for second-level domains within the TLD (e.g., .com, .net).
- **Authoritative Name Server:**
 - Holds the DNS records for specific domains and responds with the IP address for the requested domain name

DNS Record Types

Record Type	Description	Example
A	Maps domain to IPv4	<code>example.com → 192.0.2.1</code>
AAAA	Maps to IPv6	<code>example.com → 2001:db8::1</code>
CNAME	Alias for another domain	<code>mail.example.com → gmail.com</code>
MX	Mail exchange	<code>example.com →</code> <code>mailserver.example.com</code>
TXT	Metadata (SPF, DKIM)	<code>"v=spf1 include:_spf.google.com"</code>
NS	Delegates subdomains	<code>ns1.example.com</code>

DNS Query Process

- User enters domain name in browser
- Browser checks local DNS cache
- If not found, query goes to Recursive Resolver
- Resolver contacts Root Server, then TLD Server, then Authoritative Name Server
- IP address is returned to user

DNS in Cloud Environments

Traditional DNS vs. Cloud DNS

Aspect	Traditional DNS	Cloud DNS
Management	Manual config	API-based automation
Scale	Fixed servers	Global edge network
Updates	Slow propagation	Instant (dynamic records)
Integration	Static	Linked with load balancers, autoscaling

Cloud DNS Services

1. AWS Route 53

- Scalable, highly available DNS service
- Supports health checks, routing policies

2. Google Cloud DNS

- Private zones for internal name resolution
- Global anycast network

3. Azure DNS

- DNS zone hosting integrated with Azure VNet
- DNS forwarding between on-premise and Azure

Route 53 Example (AWS)

- **Hosted Zone:** container for domain records
- **Routing Policies:**
 - Simple
 - Weighted
 - Latency-based
 - Failover
 - Geolocation-based

Diagram: AWS Route 53 with multiple regions (latency-based routing)

Private DNS in VPCs

- Used for internal name resolution (private subnets)
- Private Hosted Zones accessible only within a VPC
- Resolves internal service names → private IPs
- Enables microservices communication without public exposure

Example:

db.internal.local → 10.0.2.15

DNS Load Balancing

Techniques:

1. **Round Robin**
 - Rotate multiple IPs per request.
2. **Weighted Round Robin**
 - Prioritize based on capacity.
3. **Geo-DNS (Latency Routing)**
 - Serve nearest region.
4. **Failover DNS**
 - Redirect on server failure.

Cloud Network Services

Key Components:

- **Virtual Routers**
 - Route traffic between subnets/VPCs
- **Firewalls/Security Groups**
 - Control inbound/outbound rules
- **Load Balancers**
 - Distribute traffic (Layer 4/7)
- **Elastic IPs**
 - Static, public IPs for cloud resources
- **NAT Gateways**
 - Allow private instances to reach the internet

Multihoming

- Connecting an organization's network to multiple ISPs for redundancy and performance.
- **Goal:**
 - Ensure continuous connectivity even if one link fails.
- **Approaches:**
 - **Active/Active:**
 - Both links carry traffic
 - **Active/Passive:**
 - One link for backup

Benefits of Multihoming

- **Fault Tolerance**
 - Backup path available if primary fails
- **Load Distribution**
 - Balances outgoing traffic between ISPs
- **Performance Optimization**
 - Choose lowest latency route
- **Resilience**
 - Avoid single point of failure
- **Control**
 - Fine-grained traffic policies (BGP routing)

Routing Techniques in Multihoming

Key Technique: BGP (Border Gateway Protocol)

- Dynamic route advertisement between ISPs.
- Autonomous Systems (AS) exchange reachability.
- Supports failover, path selection, and load balancing.

Other Mechanisms:

- Policy-based routing
- Weighted route preferences
- Anycast addressing

Multihoming in Cloud

- **Use Cases:**
 - **Multi-Region Deployments:**
 - Traffic routed across continents.
 - **Hybrid Networking:**
 - On-prem + Cloud link via VPN or Direct Connect.
 - **Cloud-to-Cloud Redundancy:**
 - AWS ↔ Azure peering for DR.
- **Challenges:**
 - Routing complexity
 - IP conflicts
 - Bandwidth cost
 - Policy coordination

Case Study: Netflix Global Architecture

- **Multi-Region Cloud Presence:** Deployed across AWS regions.
- **Open Connect CDN:** Edge servers near users reduce latency.
- **Global DNS Routing:** Users directed to nearest data center.
- **Active-Active Multihoming:** BGP routing + regional failover.

Diagram:

World map showing distributed Netflix nodes + DNS-based routing.

Netflix's reliability (99.99% uptime) comes from multihoming and DNS-based global traffic control.

Thank you